

ACRME — SKU-Level Capacity Reservations & Quota Groups: Design Analysis

What it would take to move the mockup (and the engine model) from a single capacity/quota number per region to SKU-grain reservations, quota-group availability during allocation, and a post-freeze “logical lock” per environment.

Field	Value
Author	ACRME design (Vishnuvardhan Reddy)
Status	Design analysis / feasibility — what-if, not a baseline change
Reconciled to	Requirements Baseline v2.4 (11 Sep 2026 amendment)
Scope	Region-selection mockup (ac-rme_region_selection_whatif_acrme_regions.xlsx) + engine data-model implications
Grounding data	SKU_USage (1).xlsx , sku_usage_prod_v1.xlsx (40,039 source rows; 169 distinct SKUs; 14 regions)

0. TL;DR

- The **mockup** abstracts capacity and quota to **one number per environment per region** (Prod Reserved / Prod Alloc / Prod Free , etc.). That was a deliberate simplification so the region-selection scoring stays readable — it is **not** what the baseline says.
- The **baseline already requires SKU grain**. CAP-022 (seed matrix of SKU × region × AZ), CAP-023 (per-AZ CRG per environment), CAP-011/012 (zone/region isolation), QUA-002 (quota by VM/quota family), and QUA-003/004 (Azure **quota groups**) mean the production engine is already SKU- and family-scoped. So this is mostly **surfacing existing requirements in the mockup**, not inventing new ones.
- The one genuinely new idea you’re asking for is **“availability by group during allocation”** — treating each placement decision as gated by the minimum of (a) the specific SKU’s reserved-free at its CRG and (b) the SKU’s **quota-family** availability in the quota group. This is the operational join of CAP-016 + QUA-007 + RDY-003, made explicit and computable.
- The **“logical lock” after a region freeze** is a new mockup capability but a natural extension of CAP-003 / CAP-016 / QUA-008: on freeze, commit the requested cores against the SKU’s reserved-free and the family’s quota-available, recompute headroom, and mark the environment **LOCKED** so subsequent what-ifs see the depleted state.

- Net effort: **mockup = medium** (5 new sheets, SUMIFS/INDEX-MATCH, one freeze input). **Engine/requirements = mostly already covered**; a handful of open questions (Section 10) need your ruling before it becomes authoritative.

1. Current state — how the mockup abstracts capacity & quota

The `Capacity_Usage` sheet carries **one row per region** with three environment blocks, each collapsed to a single scalar:

Grain today	Columns
Prod (per region)	Prod Reserved (F) , Prod Alloc (G) , Prod Free (H) , Prod Q-Limit (I) , Prod Q-Used (J) , Prod Q-Headroom (K)
NonProd (per region)	NP Reserved (L) , NP Alloc (M) , NP Eff-Free (N) , NP Q-Limit (O) , NP Q-Used (P) , NP Q-Headroom (Q)
DR (per region)	DR Reserved (R) , DR Free (S) , DR Coverage (T)

The five placement-scoring components ($\alpha \dots \epsilon$) then read these scalars. **Every SKU in a region is blended into one pool.** That is fine for ranking regions but it hides three things the baseline treats as first-class:

1. A region can have plenty of total free cores yet be unable to place a **specific SKU** (its CRG is full even though another SKU's CRG is empty) — CAP-011/CAP-023.
2. Quota is not per-region-scalar; it is **per VM/quota family** and can be **pooled across subscriptions** in an Azure **quota group** — QUA-002/003/004.
3. Reservation availability and quota availability are **two different control planes** that must both pass — QUA-001, QUA-007, RDY-003.

2. Baseline reality — SKU grain is already mandated

Baseline req	What it already says	Mockup gap
CAP-022	Managed set is SKU/VM-family × region × availability-zone, seeded at count-0	Mockup has no SKU axis
CAP-023	One regional CRG + one CRG per AZ per environment (crg-pr-eus2-az1 ...)	Mockup has no CRG / AZ axis for reservations
CAP-011 / CAP-012	Reservations tracked & isolated by region and zone ; zone-1 capacity ≠ available in zone-2	Mockup free-pool is region-level only
CAP-016	Pre-deploy validation checks SKU matches, zone matches, sufficient reserved capacity	Mockup has no per-SKU reserved check
CAP-018	Distinguish guaranteed allocated vs associated-but-unguaranteed	Mockup has a single Alloc number
QUA-002	Quota inventory by subscription, region, VM/ quota family	Mockup quota is a region scalar
QUA-003	Azure quota groups : unused family quota pooled centrally for fast reallocation	Not modelled
QUA-004	Prefer one governed quota group per regional/quota-family scope	Not modelled
QUA-007 / RDY-003	Quota ≥ reservation for any SKU we intend to scale; correlate by region/zone/ SKU/ family	Mockup can't express the join
RDY-001 / RDY-002	Readiness gate + machine states (QUOTA_DEFICIT , RESERVATION_DEFICIT , ...)	Mockup has no per-SKU readiness state

Conclusion: adding SKU-level reservations and quota groups to the mockup is **catch-up to the baseline**, not a requirements change. The what-if is really: “show the model at the grain the engine already operates on.”

3. The crucial concept: SKU ↔ quota-family (they are different grains)

This is the single most important modelling point, and the reason capacity and quota **cannot** share one axis.

- A **capacity reservation** is per **SKU** (e.g., `Standard_D8ads_v5`) in a specific **region + zone + environment** CRG.
- **Quota** is per **VM/quota family** (e.g., `Standard Dadsv5 Family vCPUs`) per **subscription + region** — **many SKUs roll up to one family**.

Worked from your real data:

SKU (reservation grain)	vCPU/instance	Quota family (quota grain)
<code>Standard_D2ads_v5</code>	2	Dadsv5
<code>Standard_D4ads_v5</code>	4	Dadsv5
<code>Standard_D8ads_v5</code>	8	Dadsv5
<code>Standard_D16ads_v5</code>	16	Dadsv5
<code>Standard_E4ads_v5</code>	4	Eadsv5
<code>Standard_E16ads_v5</code>	16	Eadsv5
<code>Standard_E32ads_v5</code>	32	Eadsv5
<code>Standard_E64ads_v5</code>	64	Eadsv5

So three D-series reservations (`D4/D8/D16`) all draw down the **same Dadsv5 quota pool**, while E-series reservations draw down **Eadsv5**. An allocation therefore has **two independent constraints**:

```
can_place(SKU, cores, region, zone, env) =
    SKU_reserved_free(CRG for SKU × region × zone × env) ≥ cores      (capacity:
CAP-016)
    AND family_quota_available(family(SKU) × region × subscription) ≥ cores (quota:
QUA-007)
```

“Availability by group” = the **minimum** of those two, because either can be the binding constraint:

```
group_availability(SKU) = MIN( SKU_reserved_free , family_quota_available )
```

A quota group (QUA-003/004) makes `family_quota_available` a **pooled** number across the subscriptions that contribute unused quota, not just the deploying subscription's default ~350 vCPU.

4. Target data model (the grain change)

4.1 Capacity grain (reservations)

```
(subscription, region, availability_zone, environment, SKU) →
reserved, allocated, associated, reserved_free = reserved - allocated, buffer, target = allocated + buffer (CAP-003), crg_name (CAP-023).
```

4.2 Quota grain (quota groups)

```
(quota_group, region, quota_family, environment_pool) →
group_limit, group_used, group_available = limit - used, pooled_contributions,
pending_increases (QUA-002/003).
```

Note the deliberate asymmetry: **capacity is zone-scoped** (CAP-011) but **quota is region-scoped, not zonal** — Azure quota has no zone dimension. The join happens at region + family.

4.3 The allocation join (what makes it usable)

For any candidate (SKU, region, zone, env) :

```
reserved_free      = SUMIFS(Capacity_By_SKU.reserved_free, SKU, region, zone, env)
family             = VLOOKUP(SKU, SKU_Family_Map)
family_quota_avail = SUMIFS(Quota_Groups.group_available, family, region, env_pool)
group_availability = MIN(reserved_free, family_quota_avail)
readiness_state    = f(reserved_free, family_quota_avail) // RDY-002 enum
```

5. “Use availability by group during allocation” — the algorithm

Placement request: {SKU, cores_needed, environment, region, zone, subscription}.

1. **Resolve family** — `family = SKU_Family_Map[SKU]` (Section 3).
2. **Capacity check (CAP-016)** — find the CRG for `SKU × region × zone × env` ;
`reserved_free = reserved - allocated` . If `reserved_free < cores_needed` → candidate is `RESERVATION_DEFICIT` (unless CAP-018 over-allocation policy allows association beyond reserved).
3. **Quota check (QUA-007)** — `family_quota_available` for `family × region` in the governing quota group (pooled per QUA-003/004). If `< cores_needed` → `QUOTA_DEFICIT` .
4. **Group availability** — `group_availability = MIN(reserved_free, family_quota_available)` . This is the number that actually governs the placement.
5. **Readiness state (RDY-002)** — emit `READY` , `READY_WITH_RISK` (e.g., quota ok but reserved needs a CAP-007 scale-up), `QUOTA_DEFICIT` , `RESERVATION_DEFICIT` , `CAPACITY_UNAVAILABLE` , or `STALE_STATE` (RDY-004 freshness).
6. **Feed region scoring** — the α (capacity headroom) component becomes **per-SKU group-availability**, not a blended region scalar, so region ranking reflects the SKU actually being placed.

Over-allocation (CAP-018): if policy permits association beyond reserved, `can_associate = reserved_free + over_alloc_allowance`, but `guaranteed = reserved_free`; the ledger keeps the two distinct.

6. Post-freeze “logical lock” — the mechanism you asked for

“Once deployment freezes to a region, I want to look at updated capacity and quota so that we are logically locked for that environment.”

Definition. A **freeze** is the moment placement is committed for an environment to a chosen region/zone/SKU. A **logical lock** commits the requested cores against the model immediately, so every later what-if reads the depleted state — **before** (or independent of) the physical Azure reservation being scaled (CAP-002 keeps Azure-first for the physical change; the logical lock is the accounting reservation).

On freeze {SKU, cores, region, zone, env, subscription} :

Step	Effect	Requirement anchor
1. Commit capacity	<code>allocated += cores</code> → <code>reserved_free -= cores</code> for that SKU’s CRG	CAP-003, CAP-016
2. Commit quota	<code>group_used += cores</code> → <code>family_quota_available -= cores</code>	QUA-007, QUA-008
3. Recompute headroom	Recompute α headroom, β quota headroom, buffer deficit (<code>target - reserved</code>)	CAP-003, CAP-007
4. Set state	Environment row → <code>LOCKED</code> ; write before/after to an Allocation_Ledger (deterministic replay)	RDY-002, OBS-005
5. Flag scale-up	If <code>reserved_free now < buffer</code> , raise <code>SCALE_UP_REQUIRED</code> (physical Azure step, CAP-007)	CAP-007

Lock semantics — two layers (keep them distinct):

- **Logical lock** = accounting commitment inside ACRME (this what-if). Instant, reversible in the model.
- **Physical reservation** = the Azure CRG quantity change. Governed by CAP-002 (Azure-first) and CAP-005/007 reconciliation. The logical lock **drives** the physical target but is not the same object.

Result view. After freeze, the `Capacity_Usage` (SKU grain) and `Quota_Groups` sheets show the **post-lock** numbers, and re-running region selection for the next environment sees the reduced availability — so, e.g., placing Prod first correctly shrinks what DR can later claim in the same region (ENV-003 keeps their CRGs separate, but the quota family pool is shared per QUA-004).

7. What it would take — mockup changes (concrete, buildable spec)

Five new/changed sheets on top of the existing workbook:

Sheet	Purpose	Key columns / formulas
SKU_Catalogue	The managed SKU set (seed matrix view, CAP-022)	SKU , vCPU/instance , Quota Family , Zonal? (CAP-020) , Eligible?
SKU_Family_Map	SKU → quota family lookup (Section 3)	SKU , Family (drives all VLOOKUPS)
Capacity_By_SKU	Reservations at region × zone × env × SKU grain (CAP-023)	CRG , Reserved , Allocated , Reserved_Free , Buffer , Target=Alloc+Buffer
Quota_Groups	Quota at group × region × family × env-pool grain (QUA-002/003/004)	Quota Group , Family , Limit , Used , Available , Pooled
Allocation_Ledger	Freeze log + logical-lock before/after (Section 6)	Timestamp , Env , Region/Zone , SKU , Cores , Res-Free_before/after , Quota_before/after , State

Wiring:

- `group_availability = MIN(SUMIFS(Capacity_By_SKU[Reserved_Free], ...), SUMIFS(Quota_Groups[Available], family,...))`.
- A **FREEZE? input cell + committed-cores cell** per environment; when set, the `_after` columns subtract the committed cores and the environment status flips to **LOCKED** (Excel: helper columns + `IF` , or a tiny “apply freeze” macro/rebuild in the builder script — keeping it formula-driven avoids macros).
- Region-scoring α reads **group_availability for the requested SKU** instead of the blended region free-pool.

Effort: ~medium. The builder script (`build_acrme_whatif_acrme_regions.py`) already generates the workbook programmatically, so these sheets are additive Python — no manual Excel surgery. The freeze/lock can be modelled as a **parameterised rebuild** (set the frozen placement, regenerate → see locked state), which is deterministic and matches how the rest of the mockup is produced.

8. What it would take — engine / requirements implications

Area	Status	Net-new work
SKU × region × AZ reservation grain	Already required (CAP-022/023)	None — surface it
Quota by family + quota groups	Already required (QUA-002/003/004)	None — surface it
Availability-by-group (MIN join)	Implied by CAP-016 + QUA-007 + RDY-003	Make the <code>MIN()</code> join explicit in the calc reference and scoring
Per-SKU readiness states	Already required (RDY-002)	Compute per candidate SKU, not per region
Logical lock on freeze	New surface , natural extension of CAP-003/008 + QUA-008	Define the lock event + ledger; distinguish logical vs physical (CAP-002)
α scoring at SKU grain	Change	α headroom = <code>group_availability(requested SKU)</code>

So the requirements delta is small and mostly editorial — the big lift is the mockup catching up plus **one new concept to ratify**: the logical-lock event and its relationship to the physical Azure reservation.

9. Open issues / gaps — need your ruling before this is authoritative

Flagged rather than assumed (consistent with prior `total_customers` handling):

- Quota-group ↔ environment scope.** QUA-004 prefers **one** governed quota group covering **prod + non-prod + DR together** (max flexibility), but ENV-003 forbids **capacity** sharing across prod/DR. Confirm the intended split: quota pooled across environments while reservations stay separate — the model assumes this, but it should be decided, not inferred.
- SKU → quota-family mapping source.** The map in Section 3 is derived from SKU naming (`Dads-v5` , `Eadsv5`). Is there an authoritative Azure family table we should ingest, or is naming-derived acceptable for the mockup? (169 distinct SKUs in the data.)
- Logical lock vs Azure-first (CAP-002).** Does a freeze **immediately** trigger the physical Azure reservation scale-up, or only the logical accounting lock with physical reconciliation on the next CAP-005/006 cycle? This determines whether `LOCKED` implies a real cost commitment.
- Over-allocation at freeze (CAP-018).** On freeze, do we lock **guaranteed = reserved_free** only, or allow committing into the over-allocation allowance? Affects the `_after` math.

5. **Zone selection within the frozen region.** Capacity is zonal (CAP-011); a freeze must pick a **zone** (or regional CRG for non-zonal SKUs, CAP-023). Confirm whether the mockup freezes at region-level (engine picks zone) or the user freezes a specific zone.
 6. **Consumer-subscription quota (QUA-013, POC-gated).** Quota is assumed to live at subscription level, not the reservation group — still POC-validation-required. The quota-group model should keep this flagged.
 7. (Carried over) `total_customers` in the γ distribution term remains undefined in v2.4 — unaffected by this change but still open.
-

10. Recommended phased plan

- **Phase 1 — Surface SKU grain (mockup).** Add `SKU_Catalogue`, `SKU_Family_Map`, `Capacity_By_SKU`, `Quota_Groups`; wire `group_availability = MIN(reserved_free, family_quota_available)`. Deliverable: read-only SKU-grain view.
 - **Phase 2 — Availability-by-group in scoring.** Point α at per-SKU group-availability; emit RDY-002 readiness per candidate. Deliverable: SKU-aware region ranking.
 - **Phase 3 — Freeze & logical lock.** Add `Allocation_Ledger` + a parameterised freeze; show before/after and `LOCKED` state; recompute for the next environment. Deliverable: the locked-state view you asked for.
 - **Phase 4 — Ratify.** Resolve Section 9 open issues; fold the confirmed decisions into the Calculation Logic Reference and (if you choose) the baseline.
-

This document is a what-if / feasibility analysis. It does not modify the v2.4 baseline; it reconciles a proposed mockup enhancement to existing requirements and lists the decisions needed to make it authoritative.